

数据结构是数据项的结构化集合，其结构性表现为数据项之间的相互联系及作用，也可以理解为定义于数据项之间的某种逻辑次序。根据这种逻辑次序的复杂程度，大致可以将各种数据结构划分为线性结构、半线性结构与非线性结构三大类。在线性结构中，各数据项按照一个线性次序构成一个整体。最为基本的线性结构统称为序列（**sequence**），根据其中数据项的逻辑次序与其物理存储地址的对应关系不同，又可进一步地将序列区分为向量（**vector**）和列表（**list**）。在向量中，所有数据项的物理存放位置与其逻辑次序完全吻合，此时的逻辑次序也称作秩（**rank**）；而在列表中，逻辑上相邻的数据项在物理上未必相邻，而是采用间接定址的方式通过封装后的位置（**position**）相互引用。

本章的讲解将围绕向量结构的高效实现而逐步展开，包括其作为抽象数据类型的接口规范以及对应的算法，尤其是高效维护动态向量的技巧。此外，还将针对有序向量，系统介绍经典的查找与排序算法，并就其性能做一分析对比，这也是本章的重点与难点所在。最后，还将引入复杂度下界的概念，并通过建立比较树模型，针对基于比较式算法给出复杂度下界的统一界定方法。

§ 2.1 从数组到向量

2.1.1 数组

C、C++和Java等程序设计语言，都将数组作为一种内置的数据类型，支持对一组相关元素的存储组织与访问操作。具体地，若集合S由n个元素组成，且各元素之间具有一个线性次序，则可将它们存放于起始于地址A、物理位置连续的一段存储空间，并统称作数组（**array**），通常以A作为该数组的标识。具体地，数组A[]中的每一元素都唯一对应于某一下标编号，在多数高级程序设计语言中，一般都是从0开始编号，依次是0号、1号、2号、...、n-1号元素，记作：

$$A = \{ a_0, a_1, \dots, a_{n-1} \} \text{ 或者} \\ A[0, n) = \{ A[0], A[1], \dots, A[n-1] \}$$

其中，对于任何 $0 \leq i < j < n$ ，A[i]都是A[j]的前驱（**predecessor**），A[j]都是A[i]的后继（**successor**）。特别地，对于任何 $i \geq 1$ ，A[i-1]称作A[i]的直接前驱（**immediate predecessor**）；对于任何 $i \leq n-2$ ，A[i+1]称作A[i]的直接后继（**immediate successor**）。任一元素的所有前驱构成其前缀（**prefix**），所有后继构成其后缀（**suffix**）。

采用这一编号规范，不仅可以使得每个元素都通过下标唯一指代，而且可以使我们直接访问到任一元素。这里所说的“访问”包含读取、修改等基本操作，而“直接”则是指这些操作都可以在常数时间内完成——只要从数组所在空间的起始地址A出发，即可根据每一元素的编号，经过一次乘法运算和一次加法运算，获得待访问元素的物理地址。具体地，若数组A[]存放空间的起始地址为A，且每个元素占用s个单位的空间，则元素A[i]对应的物理地址为：

$$A + i \times s$$

因其中元素的物理地址与其下标之间满足这种线性关系，故亦称作线性数组（**linear array**）。

2.1.2 向量

按照面向对象思想中的数据抽象原则，可对以上的数组结构做一般性推广，使得其以上特性更具普遍性。向量（**vector**）就是线性数组的一种抽象与泛化，它也是由具有线性次序的一组元素构成的集合 $V = \{ v_0, v_1, \dots, v_{n-1} \}$ ，其中的元素分别由秩相互区分。

各元素的秩（**rank**）互异，且均为 $[0, n)$ 内的整数。具体地，若元素 e 的前驱元素共计 r 个，则其秩就是 r 。以前介绍的线性递归为例，运行过程中所出现过的所有递归实例，按照相互调用的关系可构成一个线性序列。在此序列中，各递归实例的秩反映了它们各自被创建的时间先后，每一递归实例的秩等于早于它出现的实例总数。反过来，通过 r 亦可唯一确定 $e = v_r$ 。这是向量特有的元素访问方式，称作“循秩访问”（**call-by-rank**）。

经如此抽象之后，我们不再限定同一向量中的各元素都属于同一基本类型，它们本身可以是来自于更具一般性的某一类的对象。另外，各元素也不见得同时具有某一数值属性，故而并不保证它们之间能够相互比较大小。

以下首先从向量最基本的接口出发，设计并实现与之对应的向量模板类。然后在元素之间具有大小可比性的假设前提下，通过引入通用比较器或重载对应的操作符明确定义元素之间的大小判断依据，并强制要求它们按此次序排列，从而得到所谓有序向量，并介绍和分析此类向量的相关算法及其针对不同要求的各种实现版本。

§ 2.2 接口

2.2.1 ADT接口

作为一种抽象数据类型，向量对象应支持如下操作接口。

表2.1 向量ADT支持的操作接口

操 作 接 口	功 能	适 用 对 象
size()	报告向量当前的规模（元素总数）	向量
get(r)	获取秩为r的元素	向量
put(r, e)	用e替换秩为r元素的数值	向量
insert(r, e)	e作为秩为r元素插入，原后继元素依次后移	向量
remove(r)	删除秩为r的元素，返回该元素中原存放的对象	向量
disordered()	判断所有元素是否已按非降序排列	向量
sort()	调整各元素的位置，使之按非降序排列	向量
find(e)	查找等于e且秩最大的元素	向量
search(e)	查找目标元素e，返回不大于e且秩最大的元素	有序向量
deduplicate()	剔除重复元素	向量
uniquify()	剔除重复元素	有序向量
traverse()	遍历向量并统一处理所有元素，处理方法由函数对象指定	向量

以上向量操作接口，可能有多种具体的实现方式，计算复杂度也不尽相同。而在引入秩的概念并将外部接口与内部实现分离之后，无论采用何种具体的方式，符合统一外部接口规范的任一实现均可直接地相互调用和集成。